



Program Studi Matematika  
Fakultas Sains  
Institut Teknologi Sumatera

---

---

# Modul Praktikum Pemodelan Komputasi dan Analisis Numerik

MA25-31017

Disusun oleh

Rifky Fauzi

Dear Michiko Mutiara Noor

---

---

Lampung Selatan, 2026

# Daftar Isi

|                                                      |    |
|------------------------------------------------------|----|
| Daftar Isi                                           | 2  |
| Identitas Modul                                      | 5  |
| Petunjuk Umum Praktikum                              | 7  |
| 1 Keterulangan Komputasi dan Dasar MATLAB            | 9  |
| 1.1 Materi singkat                                   | 9  |
| 1.2 Perintah MATLAB dasar                            | 10 |
| 1.3 Contoh skrip utama dan fungsi terpisah           | 10 |
| 1.4 Plotting dasar                                   | 11 |
| 1.5 Latihan                                          | 11 |
| 2 Praktikum 1: Akar Persamaan Nonlinear              | 13 |
| 2.1 Materi singkat dan landasan teori                | 13 |
| 2.2 Algoritma                                        | 14 |
| 2.2.1 Biseksi                                        | 14 |
| 2.2.2 Newton-Raphson                                 | 14 |
| 2.3 Contoh                                           | 15 |
| 2.4 Program MATLAB                                   | 15 |
| 2.5 Latihan                                          | 17 |
| 3 Praktikum 2: Sistem Persamaan Linear Iteratif      | 18 |
| 3.1 Materi singkat dan landasan teori                | 18 |
| 3.2 Algoritma                                        | 19 |
| 3.3 Contoh                                           | 19 |
| 3.4 Program MATLAB                                   | 19 |
| 3.5 Latihan                                          | 20 |
| 4 Praktikum 3: Sistem Nonlinear dan Pencocokan Kurva | 22 |

|     |                                                                         |    |
|-----|-------------------------------------------------------------------------|----|
| 4.1 | Materi singkat dan landasan teori . . . . .                             | 22 |
| 4.2 | Algoritma . . . . .                                                     | 23 |
| 4.3 | Contoh . . . . .                                                        | 23 |
| 4.4 | Program MATLAB . . . . .                                                | 23 |
| 4.5 | Latihan . . . . .                                                       | 24 |
| 5   | Praktikum 4: Turunan Numerik dengan Beda Hingga . . . . .               | 26 |
| 5.1 | Materi singkat dan landasan teori . . . . .                             | 26 |
| 5.2 | Algoritma . . . . .                                                     | 27 |
| 5.3 | Contoh . . . . .                                                        | 27 |
| 5.4 | Program MATLAB . . . . .                                                | 27 |
| 5.5 | Latihan . . . . .                                                       | 28 |
| 6   | Praktikum 5: Ukuran Langkah, Galat Pemotongan, dan Richardson . . . . . | 29 |
| 6.1 | Materi singkat dan landasan teori . . . . .                             | 29 |
| 6.2 | Algoritma . . . . .                                                     | 30 |
| 6.3 | Contoh . . . . .                                                        | 30 |
| 6.4 | Program MATLAB . . . . .                                                | 30 |
| 6.5 | Latihan . . . . .                                                       | 30 |
| 7   | Praktikum 6: Integral Numerik Dasar . . . . .                           | 32 |
| 7.1 | Materi singkat dan landasan teori . . . . .                             | 32 |
| 7.2 | Algoritma . . . . .                                                     | 33 |
| 7.3 | Contoh . . . . .                                                        | 33 |
| 7.4 | Program MATLAB . . . . .                                                | 33 |
| 7.5 | Latihan . . . . .                                                       | 34 |
| 8   | Praktikum 7: Titik Tengah dan Romberg . . . . .                         | 35 |
| 8.1 | Materi singkat dan landasan teori . . . . .                             | 35 |
| 8.2 | Algoritma Romberg . . . . .                                             | 35 |
| 8.3 | Contoh . . . . .                                                        | 36 |
| 8.4 | Program MATLAB . . . . .                                                | 36 |
| 8.5 | Latihan . . . . .                                                       | 37 |
| 9   | Praktikum 8: Masalah Nilai Awal - Euler dan Runge-Kutta . . . . .       | 38 |
| 9.1 | Materi singkat dan landasan teori . . . . .                             | 38 |
| 9.2 | Contoh model . . . . .                                                  | 39 |
| 9.3 | Algoritma . . . . .                                                     | 39 |

|      |                                                                   |    |
|------|-------------------------------------------------------------------|----|
| 9.4  | Program MATLAB . . . . .                                          | 39 |
| 9.5  | Latihan . . . . .                                                 | 40 |
| 10   | Praktikum 9: Stabilitas, Ukuran Langkah, dan Reproduksi Model PDB | 41 |
| 10.1 | Materi singkat dan landasan teori . . . . .                       | 41 |
| 10.2 | Contoh . . . . .                                                  | 42 |
| 10.3 | Program MATLAB . . . . .                                          | 42 |
| 10.4 | Latihan . . . . .                                                 | 42 |
| 11   | Praktikum 10: Praktikum Terpadu, Verifikasi, dan Validasi         | 44 |
| 11.1 | Materi singkat: dari metode ke alur kerja . . . . .               | 44 |
| 11.2 | Contoh terpadu: model logistik . . . . .                          | 45 |
| 11.3 | Algoritma alur kerja . . . . .                                    | 45 |
| 11.4 | Program MATLAB . . . . .                                          | 46 |
| 11.5 | Latihan terpadu . . . . .                                         | 46 |
|      | Rubrik Analisis Umum                                              | 47 |
|      | Daftar File MATLAB                                                | 48 |
|      | Referensi Utama                                                   | 49 |



# Identitas Modul

|                       |                                                                                 |
|-----------------------|---------------------------------------------------------------------------------|
| Mata kuliah           | Pemodelan Komputasi dan Analisis Numerik                                        |
| Kode                  | MA25-31017                                                                      |
| Program studi         | Matematika                                                                      |
| Fakultas              | Fakultas Sains                                                                  |
| Institusi             | Institut Teknologi Sumatera                                                     |
| Perangkat lunak utama | MATLAB                                                                          |
| Format pembelajaran   | Praktikum, bedah jurnal, reproduksi komputasi, dan pembelajaran berbasis proyek |

## Tentang modul

Modul ini mengonsolidasikan bahan praktikum sebelumnya, materi metode numerik praktis, serta kebutuhan pelaksanaan praktikum Pemodelan Komputasi dan Analisis Numerik. Struktur praktikum diperbarui menjadi sepuluh praktikum terhitung. Elaborasi teori, contoh tambahan, pola verifikasi, dan kode MATLAB pada naskah ini disusun khusus untuk kebutuhan praktikum.

## Capaian Pembelajaran Mata Kuliah (CPMK) utama

CPMK 6.1. Mahasiswa mampu memecahkan (C4) masalah optimasi termasuk (dan tidak terbatas pada) masalah program linier, masalah pengujian hipotesis dan masalah pada proses stokastik dengan memilih (C3) model matematika yang sesuai, menerapkan (C3) langkah-langkah metode numerik yang terukur dan valid secara analitik untuk mendapatkan solusi, serta menginterpretasikan solusi yang diperoleh sesuai konteks masalah.

CPMK 7.1. Mahasiswa mampu merancang, menerapkan (C3), dan menganalisis (C4) model matematika berbasis metode numerik atau komputasi untuk memecahkan masalah di bidang sains, teknik, ekonomi, atau industri, serta mengevaluasi (C5) keakuratan, keterbatasan, dan relevansi model dalam konteks masalah nyata, dengan mendokumentasikan hasilnya secara ilmiah.



## Peta praktikum

| Praktik | Minggu | Indikator             | Fokus                                                                               |
|---------|--------|-----------------------|-------------------------------------------------------------------------------------|
| 1       | M2     | 6.1.1–6.1.3,<br>7.1.1 | Akar persamaan nonlinear: metode pengurangan dan metode terbuka                     |
| 2       | M3     | 6.1.4                 | Sistem Persamaan Linear (SPL) iteratif: Jacobi, Gauss-Seidel, residual, konvergensi |
| 3       | M4     | 6.1.5, 7.1.1          | Sistem Persamaan Nonlinear (SPNL) dan kuadrat terkecil nonlinear/pencocokan kurva   |
| 4       | M5     | 6.1.6                 | Turunan numerik: beda maju, beda mundur, beda pusat                                 |
| 5       | M6     | 6.1.7, 7.1.1          | Ukuran langkah, galat pemotongan, Richardson, reproduksi hasil                      |
| 6       | M7     | 6.1.8                 | Integral numerik: Trapezoid dan Simpson; pembandingan Titik Tengah                  |
| 7       | M9     | 6.1.9, 7.1.1          | Titik Tengah, Romberg, studi akurasi dan reproduksi artikel                         |
| 8       | M10    | 6.1.10                | Masalah nilai awal: Euler dan evaluasi galat lokal/global                           |
| 9       | M11    | 6.1.11, 7.1.1         | Runge-Kutta, stabilitas, ukuran langkah, reproduksi kurva model                     |
| 10      | M12    | 6.1/7.1               | Praktikum terpadu: pemilihan metode, verifikasi dan validasi                        |



# Petunjuk Umum Praktikum

## Struktur setiap bab

Setiap bab memuat enam komponen wajib: (1) capaian pembelajaran, (2) materi/teori, (3) algoritma, (4) contoh, (5) program MATLAB, dan (6) latihan. Naskah ini menambahkan tiga komponen yang disarankan untuk pembelajaran komputasi: target luaran, checklist analisis, dan verifikasi dan validasi.

## Prinsip pemrograman numerik

- i. Pisahkan skrip utama dan fungsi numerik. Skrip utama memuat data, parameter, visualisasi, dan analisis; fungsi memuat algoritma.
- ii. Hindari invers matriks eksplisit. Di MATLAB, gunakan operator `\` untuk menyelesaikan sistem linear.
- iii. Gunakan minimal dua kriteria berhenti ketika relevan: perubahan iterasi dan residual.
- iv. Selalu tetapkan `maxit` untuk mencegah loop tak berhingga.
- v. Laporkan satuan, toleransi, ukuran langkah, jumlah iterasi, residual, dan metrik galat.
- vi. Pisahkan verifikasi (apakah algoritma diimplementasikan benar?) dari validasi (apakah model/hasil sesuai fenomena atau data?).

## Format ringkas laporan praktikum

- i. Tujuan dan capaian yang diuji.
- ii. Model/persamaan dan asumsi.
- iii. Metode numerik serta algoritma/pseudocode.
- iv. Implementasi MATLAB yang dapat direproduksi.



- v. Verifikasi: kasus uji, residual, konvergensi, atau pembanding analitik.
- vi. Hasil: tabel dan grafik dengan label lengkap.
- vii. Analisis: pengaruh parameter numerik (toleransi, ukuran langkah, tebakan awal) dan keterbatasan.
- viii. Kesimpulan singkat berbasis hasil, bukan hanya mengulang teori.





## Bab 1

# Keterulangan Komputasi dan Dasar MATLAB

### Capaian pembelajaran praktikum

Mahasiswa memahami prinsip keterulangan komputasi, mampu menjalankan MATLAB atau MATLAB Online, mampu memisahkan skrip utama dan fungsi, serta mampu membuat visualisasi dasar untuk menganalisis hasil numerik.

### Target luaran

Folder kerja berisi satu skrip utama, beberapa fungsi terpisah, gambar hasil plot, dan catatan singkat cara menjalankan ulang program.

## 1.1 Materi singkat

Dalam praktikum komputasi, hasil yang baik harus dapat dijalankan ulang oleh orang lain dengan hasil yang sama atau sangat dekat. Prinsip ini disebut keterulangan komputasi. Kode yang dapat diulang biasanya memiliki struktur folder jelas, nama file konsisten, parameter tertulis eksplisit, dan fungsi numerik dipisahkan dari skrip utama.

Gunakan pola berikut.

- i. Skrip utama: memuat data, parameter, pemanggilan fungsi, tabel, dan plot. Contoh nama: `root_compare.m`.
- ii. Fungsi: memuat algoritma numerik yang dapat dipakai ulang. Contoh nama: `bisection_fun.m`. Nama file harus sama dengan nama fungsi.
- iii. Luaran: tabel, gambar, dan file `.mat` disimpan dengan nama yang informatif.
- iv. Catatan jalan ulang: tuliskan versi MATLAB, folder kerja, nama file utama, dan parameter yang dipakai.



MATLAB dapat digunakan melalui aplikasi desktop atau melalui MATLAB Online pada alamat <https://matlab.mathworks.com>. Setelah masuk, unggah folder kode praktikum, buka skrip utama, lalu tekan Run. Pastikan semua fungsi berada pada folder yang sama atau sudah masuk ke path MATLAB.

## 1.2 Perintah MATLAB dasar

| Perintah                                                                  | Kegunaan                                                       |
|---------------------------------------------------------------------------|----------------------------------------------------------------|
| <code>clc</code>                                                          | Membersihkan tampilan Command Window.                          |
| <code>clear</code>                                                        | Menghapus variabel dari workspace.                             |
| <code>close all</code>                                                    | Menutup semua jendela gambar.                                  |
| <code>format long</code>                                                  | Menampilkan angka dengan digit lebih banyak.                   |
| <code>whos</code>                                                         | Menampilkan daftar variabel, ukuran, dan tipe data.            |
| <code>help namaFungsi</code>                                              | Membaca bantuan singkat suatu fungsi.                          |
| <code>doc namaFungsi</code>                                               | Membuka dokumentasi MATLAB.                                    |
| <code>linspace(a,b,n)</code>                                              | Membuat vektor dari $a$ ke $b$ sebanyak $n$ titik.             |
| <code>zeros(n,m)</code> ,<br><code>ones(n,m)</code> , <code>eye(n)</code> | Membuat matriks nol, satu, dan identitas.                      |
| <code>length(x)</code> , <code>numel(x)</code> ,<br><code>size(A)</code>  | Mengecek ukuran vektor atau matriks.                           |
| <code>disp</code> , <code>fprintf</code>                                  | Menampilkan hasil ke layar.                                    |
| <code>save</code> , <code>load</code>                                     | Menyimpan dan memanggil variabel dari file <code>.mat</code> . |

## 1.3 Contoh skrip utama dan fungsi terpisah

```
% File: main_demo.m
clear; clc; close all; format long
f = @(x) exp(x) + x;
a = -1; b = 0; tol = 1e-8; maxit = 100;
[x, hist] = bisection_fun(f, a, b, tol, maxit);
fprintf('Akar=%%.12f, residual=%%.3e\n', x, abs(f(x)));
semilogy(hist.iter, hist.res, 'o-'); grid on
xlabel('Iterasi'); ylabel('Residual'); title('Konvergensi_Biseksi')

function [x, hist] = bisection_fun(f, a, b, tol, maxit)
    if f(a)*f(b) > 0
        error('Interval tidak mengurung akar')
    end
    for k = 1:maxit
        x = (a+b)/2;
        hist.iter(k,1) = k;
```



```
hist.res(k,1) = abs(f(x));  
if hist.res(k) < tol, return, end  
if f(a)*f(x) < 0, b = x; else, a = x; end  
end  
end
```

## 1.4 Plotting dasar

Visualisasi dipakai untuk membaca pola galat, konvergensi, dan kewajaran model. Gunakan label sumbu dan legenda agar grafik dapat dibaca tanpa penjelasan lisan.

```
x = linspace(0, 2*pi, 200);  
y1 = sin(x); y2 = cos(x);  
plot(x, y1, 'LineWidth', 1.5); hold on  
plot(x, y2, '—', 'LineWidth', 1.5); grid on  
xlabel('x'); ylabel('nilai_fungsi')  
title('Contoh_plot_sin_dan_cos')  
legend('sin(x)', 'cos(x)', 'Location', 'best')  
  
% Plot galat konvergensi  
semilogy(1:numel(err), err, 'o-'); grid on  
xlabel('Iterasi'); ylabel('Galat')  
  
% Plot orde terhadap ukuran langkah  
loglog(hs, errs, 's-'); grid on  
xlabel('h'); ylabel('Galat')
```

## 1.5 Latihan

- Buat folder `Praktikum_00` yang berisi `main_demo.m` dan satu fungsi terpisah. Jalankan melalui MATLAB Online.
- Buat fungsi `relative_error(xnew,xold)` untuk menghitung galat relatif iterasi.
- Buat tabel berisi  $h$ , hampiran, dan galat untuk fungsi  $f(x) = \sin x$  di  $x = 1$ .
- Buat tiga grafik: `plot`, `semilogy`, dan `loglog`. Jelaskan kapan masing-masing lebih sesuai dipakai.
- Simpan hasil perhitungan ke file `hasil_praktikum00.mat`, lalu panggil ulang dengan `load`.
- Tulis catatan jalan ulang maksimal setengah halaman: perangkat lunak, folder, file utama, fungsi, parameter, dan cara menjalankan.
- Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.



- viii. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- ix. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 2

# Praktikum 1: Akar Persamaan Nonlinear

### Capaian pembelajaran praktikum

Mahasiswa mampu menjelaskan sumber galat, akurasi, kriteria berhenti, dan konvergensi; menerapkan Biseksi, Regula Falsi, Titik Tetap, Newton-Raphson, dan Sekan; serta membandingkan karakteristik konvergensinya (indikator 6.1.1–6.1.3 dan 7.1.1).

### Target luaran

Satu skrip utama MATLAB yang membandingkan sedikitnya empat metode pencarian akar, tabel iterasi, residual, grafik konvergensi, dan interpretasi mengapa metode tertentu lebih cepat atau lebih tahan gangguan.

## 2.1 Materi singkat dan landasan teori

Masalah pencarian akar dinyatakan sebagai

$$f(x) = 0. \quad (2.1)$$

Pada model nyata, akar dapat bermakna titik impas, waktu ketika suatu ambang tercapai, temperatur kesetimbangan, atau parameter yang membuat residual sama dengan nol. Dua keluarga utama metode adalah metode pengurungan dan metode terbuka. Metode pengurungan mempertahankan interval yang mengurung akar, sedangkan metode terbuka membangun aproksimasi baru tanpa harus mempertahankan perubahan tanda.

Jika  $f$  kontinu pada  $[a, b]$  dan  $f(a)f(b) < 0$ , Teorema Nilai Antara menjamin sedikitnya satu akar di dalam interval. Kondisi ini menjadi dasar Biseksi dan Regula Falsi. Biseksi selalu mengurangi panjang interval menjadi setengah, sehingga konvergensinya linear tetapi sangat tahan gangguan. Regula Falsi memakai perpotongan garis secant melalui  $(a, f(a))$  dan  $(b, f(b))$  dengan sumbu- $x$  sehingga sering lebih cepat, walaupun salah satu ujung interval dapat tertahan terlalu lama.

Newton-Raphson dapat diperoleh dari linearisasi Taylor

$$f(x_i + \Delta x) \approx f(x_i) + f'(x_i)\Delta x = 0, \quad (2.2)$$



sehingga

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}. \quad (2.3)$$

Dekat akar sederhana, Newton biasanya memiliki konvergensi kuadratik. Namun performanya sensitif terhadap tebakan awal dan nilai turunan yang kecil. Metode Sekan menghindari evaluasi turunan dengan menggantinya menggunakan beda terbagi. Metode Titik Tetap menulis persamaan sebagai  $x = g(x)$  dan mengiterasikan  $x_{i+1} = g(x_i)$ . Syarat lokal sederhana untuk konvergensi titik tetap adalah  $|g'(x^*)| < 1$  di sekitar solusi.

Dua ukuran penting adalah residual  $|f(x_i)|$  dan perubahan iterasi. Kriteria perubahan relatif yang stabil secara numerik dapat ditulis

$$e_i = \frac{|x_i - x_{i-1}|}{\max(1, |x_i|)}. \quad (2.4)$$

Residual kecil menunjukkan persamaan hampir terpenuhi, sedangkan perubahan iterasi kecil menunjukkan proses iterasi sudah stagnan. Keduanya sebaiknya diperiksa bersama.

## 2.2 Algoritma

### 2.2.1 Biseksi

---

#### Algorithm 1 Metode Biseksi

---

```
1: Pilih  $a, b$  dengan  $f(a)f(b) < 0$ , toleransi  $\tau$ , dan  $N_{\max}$ .
2: for  $k = 1, 2, \dots, N_{\max}$  do
3:    $c \leftarrow (a + b)/2$ .
4:   if  $|f(c)| \leq \tau$  atau perubahan iterasi  $\leq \tau$  then
5:     berhenti.
6:   end if
7:   if  $f(a)f(c) < 0$  then
8:      $b \leftarrow c$ 
9:   else
10:     $a \leftarrow c$ 
11:   end if
12: end for
```

---

### 2.2.2 Newton-Raphson




---

### Algorithm 2 Metode Newton-Raphson

---

- 1: Pilih  $x_0$ , toleransi  $\tau$ , dan  $N_{\max}$ .
  - 2: for  $k = 0, 1, \dots, N_{\max} - 1$  do
  - 3:   Periksa  $|f'(x_k)|$  tidak terlalu kecil.
  - 4:    $x_{k+1} \leftarrow x_k - f(x_k)/f'(x_k)$ .
  - 5:   Hitung residual dan perubahan iterasi.
  - 6:   if kriteria berhenti terpenuhi then
  - 7:     berhenti.
  - 8:   end if
  - 9: end for
- 

## 2.3 Contoh

Tentukan akar  $e^x + x = 0$ . Grafik menunjukkan perubahan tanda pada  $[-1, 0]$ . Nilai referensi adalah

$$x^* \approx -0.567143290409784. \quad (2.5)$$

Biseksi menggunakan interval  $[-1, 0]$ , Newton dapat dimulai dari  $x_0 = 0$ , sedangkan Sekan menggunakan dua titik awal  $-1$  dan  $0$ . Hasil yang baik bukan hanya angka akhir, tetapi juga rekaman jumlah iterasi, residual, dan kurva penurunan galat.

Catatan numerik. Kecepatan konvergensi tidak sama dengan tahan gangguan. Metode yang sangat cepat dapat gagal jika tebakan awal buruk. Karena itu perbandingan metode harus menyebutkan kebutuhan informasi awal dan mode kegagalan.

## 2.4 Program MATLAB

```
root_compare.m

%% Praktikum 1: perbandingan metode pencarian akar
clear; clc; format long g
f = @(x) exp(x) + x;
df = @(x) exp(x) + 1;
g = @(x) -exp(x); % bentuk fixed-point x = g(x)
tol = 1e-8; maxit = 100;

[xb,hb] = bisection_fun(f,-1,0,tol,maxit);
[xf,hf] = regula_falsi_fun(f,-1,0,tol,maxit);
[xn,hn] = newton_fun(f,df,0,tol,maxit);
[xs,hs] = secant_fun(f,-1,0,tol,maxit);
[xg,hg] = fixedpoint_fun(g,-0.5,tol,maxit);

fprintf('Biseksi: %.12f (%d iterasi)\n',xb,height(hb));
fprintf('Regula_falsi: %.12f (%d iterasi)\n',xf,height(hf));
fprintf('Newton: %.12f (%d iterasi)\n',xn,height(hn));
fprintf('Secant: %.12f (%d iterasi)\n',xs,height(hs));
```



```
fprintf('Fixed_point: %.12f(%d_iterasi)\n',xg,height(hg));

figure; semilogy(hb.iter,hb.err,'o-'); hold on
semilogy(hf.iter,hf.err,'s-'); semilogy(hn.iter,hn.err,'^-');
semilogy(hs.iter,hs.err,'d-'); semilogy(hg.iter,hg.err,'x-');
grid on; xlabel('Iterasi'); ylabel('Error_aproksimasi');
legend('Biseksi','Regula_Falsi','Newton','Sekan','Titik_Tetap','Location','best');
```

bisection\_fun.m

```
function [x,history] = bisection_fun(f,a,b,tol,maxit)
fa=f(a); fb=f(b);
if fa*fb>0, error('Interval_awal_tidak_mengurung_akar. '); end
xold = NaN; iter=[]; xs=[]; errs=[]; res=[];
for k=1:maxit
    x=(a+b)/2; fx=f(x);
    if k==1, err=Inf; else, err=abs(x-xold)/max(1,abs(x)); end
    iter(end+1,1)=k; xs(end+1,1)=x; errs(end+1,1)=err; res(end+1,1)=abs(fx);
    if abs(fx)<=tol || err<=tol, break; end
    if fa*fx<0
        b=x; fb=fx;
    else
        a=x; fa=fx;
    end
    xold=x;
end
history=table(iter,xs,errs,res,'VariableNames',{'iter','x','err','residual'});
end
```

newton\_fun.m

```
function [x,history] = newton_fun(f,df,x0,tol,maxit)
iter=[]; xs=[]; errs=[]; res=[];
for k=1:maxit
    d=df(x0);
    if abs(d)<sqrt(eps), error('Turunan_terlalu_kecil. '); end
    x=x0-f(x0)/d;
    err=abs(x-x0)/max(1,abs(x));
    iter(end+1,1)=k; xs(end+1,1)=x; errs(end+1,1)=err; res(end+1,1)=abs(f(x));
    if res(end)<=tol || err<=tol, break; end
    x0=x;
end
history=table(iter,xs,errs,res,'VariableNames',{'iter','x','err','residual'});
end
```

Fungsi regula\_falsi\_fun.m, secant\_fun.m, dan fixedpoint\_fun.m tersedia pada paket kode.

Checklist analisis. (i) Apakah residual dan galat iterasi keduanya turun? (ii) Metode mana yang paling cepat? (iii) Apakah perubahan tebakan awal mengubah keberhasilan Newton/Sekan/Titik Tetap? (iv) Apakah interval Biseksi benar-benar mengurung akar?





## 2.5 Latihan

- i. Selesaikan  $x^3 - 4x - 9 = 0$  dengan Biseksi, Regula Falsi, Newton, dan Sekan sampai toleransi  $10^{-8}$ . Bandingkan jumlah iterasi.
- ii. Untuk  $f(x) = \cos x - x$ , uji dua bentuk titik tetap yang berbeda. Jelaskan mengapa salah satunya dapat lebih stabil.
- iii. Ubah tebakan awal Newton pada  $x^3 - 2x + 2 = 0$ . Dokumentasikan tebakan yang konvergen dan yang gagal/berosilasi.
- iv. Buat grafik  $\log(e_k)$  terhadap iterasi. Gunakan pola grafik untuk membedakan konvergensi linear dan lebih cepat dari linear.
- v. Cari satu artikel yang memakai pencarian akar. Identifikasi persamaan yang dicari akarnya, metode, toleransi, dan bagaimana penulis memvalidasi akar tersebut.
- vi. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vii. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- viii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 3

# Praktikum 2: Sistem Persamaan Linear Iteratif

### Capaian pembelajaran praktikum

Mahasiswa mampu menyelesaikan sistem persamaan linear secara iteratif dengan Jacobi dan Gauss-Seidel serta mengevaluasi residual dan konvergensi (indikator 6.1.4).

### Target luaran

Implementasi Jacobi dan Gauss-Seidel untuk SPL berdimensi umum, kurva residual, serta analisis kondisi yang membuat iterasi konvergen atau lambat.

### 3.1 Materi singkat dan landasan teori

Sistem persamaan linear dinyatakan  $A\mathbf{x} = \mathbf{b}$ . Metode langsung mencari solusi dalam jumlah operasi terbatas (dalam aritmetika eksak), sedangkan metode iteratif membangun barisan  $\mathbf{x}^{(k)}$  yang diharapkan menuju solusi. Metode iteratif penting untuk sistem besar dan sparse, serta menjadi dasar untuk banyak penyelesaian numerik pada pemodelan komputasi.

Pisahkan  $A = D + L + U$ , dengan  $D$  diagonal,  $L$  bagian segitiga bawah, dan  $U$  bagian segitiga atas. Jacobi menggunakan seluruh nilai iterasi lama:

$$\mathbf{x}^{(k+1)} = D^{-1} \left( \mathbf{b} - (L + U)\mathbf{x}^{(k)} \right). \quad (3.1)$$

Gauss-Seidel langsung menggunakan komponen baru yang sudah tersedia pada iterasi yang sama. Dalam bentuk matriks,

$$(D + L)\mathbf{x}^{(k+1)} = \mathbf{b} - U\mathbf{x}^{(k)}. \quad (3.2)$$

Secara komputasi, bentuk tersebut tidak perlu diimplementasikan dengan invers. Gunakan operasi skalar per baris atau penyelesaian sistem segitiga.

Residual didefinisikan

$$\mathbf{r}^{(k)} = A\mathbf{x}^{(k)} - \mathbf{b}. \quad (3.3)$$



Norma residual mengukur seberapa baik solusi aproksimasi memenuhi persamaan asli. Konvergensi iterasi linear  $\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} + \mathbf{c}$  dijamin jika radius spektral  $\rho(B) < 1$ . Diagonal dominan ketat merupakan syarat cukup yang mudah diperiksa untuk banyak kasus Jacobi/Gauss-Seidel, walaupun bukan syarat perlu.

### 3.2 Algoritma

---

#### Algorithm 3 Jacobi untuk $A\mathbf{x} = \mathbf{b}$

---

- 1: Tetapkan  $\mathbf{x}^{(0)}$ ,  $\tau$ ,  $N_{\max}$ .
  - 2: for  $k = 0, 1, \dots$  do
  - 3:   for  $i = 1, \dots, n$  do
  - 4:      $x_i^{(k+1)} \leftarrow (b_i - \sum_{j \neq i} a_{ij}x_j^{(k)}) / a_{ii}$ .
  - 5:   end for
  - 6:   Hitung perubahan iterasi dan  $\|A\mathbf{x}^{(k+1)} - \mathbf{b}\|$ .
  - 7:   Hentikan jika kriteria terpenuhi atau  $N_{\max}$  tercapai.
  - 8: end for
- 

### 3.3 Contoh

Gunakan

$$A = \begin{bmatrix} 3 & -0.1 & -0.2 \\ 0.1 & 7 & -0.3 \\ 0.3 & -0.2 & 10 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 7.85 \\ -19.3 \\ 71.4 \end{bmatrix}. \quad (3.4)$$

Solusi referensi adalah  $\mathbf{x} = [3, -2.5, 7]^T$ . Karena matriks dominan diagonal, kedua metode diharapkan konvergen dari tebakan nol. Pada contoh ini Gauss-Seidel biasanya membutuhkan lebih sedikit iterasi karena memakai informasi yang baru diperbarui.

### 3.4 Program MATLAB

```
%% Praktikum 2: Jacobi dan Gauss-Seidel
clear; clc; format long g
A=[3 -0.1 -0.2; 0.1 7 -0.3; 0.3 -0.2 10];
b=[7.85; -19.3; 71.4];
x0=zeros(size(b)); tol=1e-10; maxit=200;
[xj,hj]=jacobi_iter(A,b,x0,tol,maxit);
[xg,hg]=gauss_seidel_iter(A,b,x0,tol,maxit);

fprintf('Jacobi %12.8f\n',xj); fprintf('\n');
fprintf('Gauss-Seidel %12.8f\n',xg); fprintf('\n');
fprintf('Residual_Jacobi=%12.3e\n',norm(A*xj-b,inf));
fprintf('Residual_Gauss-Seidel=%12.3e\n',norm(A*xg-b,inf));
```



```
figure; semilogy(hj.iter,hj.residual,'o-'); hold on
semilogy(hg.iter,hg.residual,'s-'); grid on
xlabel('Iterasi'); ylabel('||Ax-b||_\infty'); legend('Jacobi','Gauss-Seidel');

function [x,history]=jacobi_iter(A,b,x0,tol,maxit)
[n,m]=size(A); if n~=m || length(b)~=n, error('Dimensi_tidak_cocok. '); end
D=diag(A); if any(abs(D)<eps), error('Diagonal_A_memuat_nol. '); end
iter=[]; err=[]; residual=[];
for k=1:maxit
    x=(b-(A-diag(D))*x0)./D;
    e=norm(x-x0,inf)/max(1,norm(x,inf)); r=norm(A*x-b,inf);
    iter(end+1,1)=k; err(end+1,1)=e; residual(end+1,1)=r;
    if e<=tol && r<=sqrt(tol), break; end
    x0=x;
end
history=table(iter,err,residual);
end

function [x,history]=gauss_seidel_iter(A,b,x0,tol,maxit)
n=length(b); x=x0; iter=[]; err=[]; residual=[];
for k=1:maxit
    xold=x;
    for i=1:n
        if abs(A(i,i))<eps, error('Diagonal_A_memuat_nol. '); end
        s1=A(i,1:i-1)*x(1:i-1);
        s2=A(i,i+1:n)*xold(i+1:n);
        x(i)=(b(i)-s1-s2)/A(i,i);
    end
    e=norm(x-xold,inf)/max(1,norm(x,inf)); r=norm(A*x-b,inf);
    iter(end+1,1)=k; err(end+1,1)=e; residual(end+1,1)=r;
    if e<=tol && r<=sqrt(tol), break; end
end
history=table(iter,err,residual);
end
```

Catatan numerik. Jangan menilai konvergensi hanya dari  $\|x^{(k+1)} - x^{(k)}\|$ . Iterasi dapat berubah sangat kecil tetapi masih memiliki residual besar pada masalah yang terskala buruk.

Checklist analisis. Periksa diagonal dominan, plot residual, bandingkan jumlah iterasi, dan uji perubahan skala baris. Laporkan norma yang digunakan.

### 3.5 Latihan

- i. Selesaikan SPL  $10x_1 + 2x_2 - x_3 = 27$ ,  $-3x_1 - 6x_2 + 2x_3 = -61.5$ ,  $x_1 + x_2 + 5x_3 = -21.5$  dengan Jacobi dan Gauss-Seidel.
- ii. Tukar urutan dua persamaan dan amati pengaruhnya terhadap konvergensi.
- iii. Bangun matriks tridiagonal ukuran  $n = 50$  dan bandingkan waktu/iterasi kedua metode.



- iv. Hitung matriks iterasi Jacobi dan radius spektralnya menggunakan `eig`. Hubungkan  $\rho(B)$  dengan kurva residual.
- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 4

# Praktikum 3: Sistem Nonlinear dan Pencocokan Kurva

### Capaian pembelajaran praktikum

Mahasiswa mampu menerapkan penyelesaian sistem nonlinear dan kuadrat terkecil/pencocokan kurva sebagai jembatan menuju pemodelan komputasi (indikator 6.1.5 dan 7.1.1).

### Target luaran

Penyelesai numerik Newton untuk sistem nonlinear dan penyelesai numerik Gauss-Newton teredam untuk kuadrat terkecil nonlinear, dilengkapi residual, grafik data-model, dan interpretasi parameter.

## 4.1 Materi singkat dan landasan teori

Sistem Persamaan Nonlinear (SPNL) ditulis

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}, \quad \mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n. \quad (4.1)$$

Newton multivariabel mengganti turunan skalar dengan matriks Jacobian  $J_F(\mathbf{x})$ . Linearisasi memberikan

$$J_F(\mathbf{x}^{(k)})\Delta\mathbf{x}^{(k)} = -\mathbf{F}(\mathbf{x}^{(k)}), \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}^{(k)}. \quad (4.2)$$

Perhatikan bahwa persamaan untuk  $\Delta\mathbf{x}$  adalah SPL. Implementasi yang baik memakai  $\text{delta} = -J \backslash F$  dan tidak menghitung  $\text{inv}(J)$ .

Pada pencocokan kurva, jumlah data biasanya lebih banyak daripada jumlah parameter. Jika  $y_i \approx f(x_i; \mathbf{p})$ , residual didefinisikan  $r_i = y_i - f(x_i; \mathbf{p})$  dan objektif kuadrat terkecil

$$S(\mathbf{p}) = \frac{1}{2} \|\mathbf{r}(\mathbf{p})\|_2^2. \quad (4.3)$$



Gauss-Newton melinearkan model terhadap parameter dan menyelesaikan

$$(J^T J) \Delta \mathbf{p} = J^T \mathbf{r}. \quad (4.4)$$

Jika  $J^T J$  berkondisi buruk atau langkah terlalu agresif, peredaman dan pelacakan balik dapat digunakan. Prinsip ini merupakan jembatan menuju optimisasi nonlinear dan kalibrasi model.

## 4.2 Algoritma

---

Algorithm 4 Newton untuk sistem nonlinear

---

- 1: Pilih  $\mathbf{x}^{(0)}$ ,  $\tau$ ,  $N_{\max}$ .
  - 2: for  $k = 0, 1, \dots$  do
  - 3:   Evaluasi  $\mathbf{F}_k$  dan  $J_k$ .
  - 4:   Selesaikan  $J_k \Delta \mathbf{x} = -\mathbf{F}_k$ .
  - 5:    $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta \mathbf{x}$ .
  - 6:   Hentikan jika  $\|\mathbf{F}(\mathbf{x}^{(k+1)})\|$  atau langkah relatif cukup kecil.
  - 7: end for
- 

## 4.3 Contoh

Untuk SPNL

$$x^2 + y^2 = 4, \quad e^x + y - 1 = 0, \quad (4.5)$$

tebakan  $[1, -1]^T$  menghasilkan salah satu solusi sekitar  $[1.00416874, -1.72963729]^T$ .

Untuk pencocokan kurva, gunakan model saturasi

$$y = \frac{ax}{b+x} \quad (4.6)$$

terhadap data  $x = [0.25, 0.75, 1.25, 1.75, 2.25]^T$  dan  $y = [0.28, 0.57, 0.68, 0.74, 0.79]^T$ . Gauss-Newton teredam memberikan parameter sekitar  $a = 1.0057$  dan  $b = 0.6077$  dengan residual kuadrat kecil.

## 4.4 Program MATLAB

```
%% Praktikum 3: SPNL dan nonlinear least-squares
clear; clc; format long g
F=@(z) [z(1)^2+z(2)^2-4; exp(z(1))+z(2)-1];
J=@(z) [2*z(1), 2*z(2); exp(z(1)), 1];
[z,h]=newton_system(F,J,[1;-1],1e-10,50);
fprintf('Solusi SPNL = [%10f %10f], residual=%3e\n',z(1),z(2),norm(F(z)));

x=[0.25;0.75;1.25;1.75;2.25];
```



```

y=[0.28;0.57;0.68;0.74;0.79];
model=@(p,x) p(1).*x./(p(2)+x);
Jm=@(p,x) [x./(p(2)+x), -p(1).*x./(p(2)+x).^2];
[p, hp]=gauss_newton_damped(model, Jm, x, y, [1;0.5], 1e-10, 100);
fprintf('Parameter_fit_a=%8f, _b=%8f, _SSE=%3e\n', p(1), p(2), sum((y-model(p,x)).^2));
xx=linspace(min(x), max(x), 200)';
figure; plot(x, y, 'o', xx, model(p, xx), '-'); grid on
xlabel('x'); ylabel('y'); legend('Data', 'Model', 'Location', 'best');

function [x, history]=newton_system(F, J, x0, tol, maxit)
x=x0(:); iter=[]; step=[]; residual=[];
for k=1:maxit
    Fx=F(x); Jx=J(x);
    delta=-Jx\Fx; xnew=x+delta;
    r=norm(F(xnew), 2); s=norm(delta, 2)/max(1, norm(xnew, 2));
    iter(end+1,1)=k; step(end+1,1)=s; residual(end+1,1)=r;
    x=xnew;
    if r<=tol || s<=tol, break; end
end
history=table(iter, step, residual);
end

function [p, history]=gauss_newton_damped(model, Jfun, x, y, p0, tol, maxit)
p=p0(:); lambda=1e-6; iter=[]; step=[]; sse=[];
for k=1:maxit
    r=y-model(p, x); J=Jfun(p, x);
    A=J'*J; g=J'*r;
    delta=(A+lambda*max(1, trace(A)/length(p))*eye(length(p)))\g;
    alpha=1; old=sum(r.^2);
    while alpha>2^-12
        pt=p+alpha*delta; new=sum((y-model(pt, x)).^2);
        if new<old, break; end
        alpha=alpha/2;
    end
    pnew=p+alpha*delta; s=norm(pnew-p)/max(1, norm(pnew));
    p=pnew; iter(end+1,1)=k; step(end+1,1)=s; sse(end+1,1)=sum((y-model(p, x)).^2);
    if s<=tol, break; end
end
history=table(iter, step, sse);
end

```

Checklist analisis. Untuk SPNL: cek norma residual dan sensitivitas terhadap tebakan awal. Untuk pencocokan kurva: cek jumlah kuadrat galat (SSE), pola residual versus  $x$ , kewajaran parameter, dan apakah parameter dapat diidentifikasi dari data.

## 4.5 Latihan

- i. Selesaikan sistem  $x^2 + y^2 = 5$  dan  $x - y = 1$  dengan Newton dari tiga tebakan awal berbeda.
- ii. Fit model eksponensial  $y = a \exp(bx)$  tanpa transformasi linear menggunakan Gauss-Newton. Bandingkan dengan hasil transformasi log.





- iii. Pada model saturasi, ubah tebakan awal  $[a, b]$  dan dokumentasikan sensitivitas penyelesaian numerik.
- iv. Cari artikel yang melakukan estimasi parameter nonlinear. Identifikasi fungsi objektif, algoritma estimasi, dan metrik kualitas kecocokan.
- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 5

# Praktikum 4: Turunan Numerik dengan Beda Hingga

Capaian pembelajaran praktikum

Mahasiswa mampu menerapkan beda maju, beda mundur, dan beda pusat serta menjelaskan orde akurasi turunan numerik (indikator 6.1.6).

Target luaran

Fungsi MATLAB generik untuk tiga skema beda hingga, tabel galat terhadap ukuran langkah, dan grafik log-log yang menunjukkan perbedaan orde akurasi.

### 5.1 Materi singkat dan landasan teori

Beda hingga diturunkan dari ekspansi Taylor. Untuk  $h > 0$ ,

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + O(h^3), \quad (5.1)$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) + O(h^3). \quad (5.2)$$

Dari persamaan pertama diperoleh beda maju

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} = f'(x) + O(h). \quad (5.3)$$

Beda mundur juga berorde satu. Dengan mengurangkan dua ekspansi Taylor diperoleh beda pusat

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} = f'(x) + O(h^2). \quad (5.4)$$

Karena itu, ketika  $h$  dibagi dua, galat pemotongan beda maju idealnya turun sekitar faktor 2, sedangkan beda pusat sekitar faktor 4, selama pembulatan belum dominan.

Ukuran langkah tidak boleh dipilih sekecil mungkin tanpa batas. Ketika  $h$  terlalu kecil, pengurangan dua bilangan yang hampir sama menyebabkan pembatalan numerik dan pembulatan



galat dapat membesar. Secara praktis terdapat wilayah dominasi galat pemotongan, wilayah optimum, dan wilayah dominasi pembulatan.

## 5.2 Algoritma

- i. Tetapkan  $f$ , titik  $x_0$ , dan daftar  $h$ .
- ii. Untuk setiap  $h$ , hitung beda maju, mundur, dan pusat.
- iii. Jika solusi analitik tersedia, hitung galat absolut/relatif. Jika tidak, gunakan solusi referensi berakurasi lebih tinggi secara hati-hati.
- iv. Plot galat terhadap  $h$  pada skala log-log.
- v. Estimasi kemiringan grafik untuk mengamati orde akurasi.

## 5.3 Contoh

Ambil

$$f(x) = \frac{e^x}{\sin(\sqrt{x})}, \quad x_0 = 1.2. \quad (5.5)$$

Turunan analitik pada  $x_0$  sekitar 2.85683778241. Dengan  $h = 0.1$ , beda pusat memberikan sekitar 2.86120081209, jauh lebih akurat daripada beda maju/mundur pada step yang sama. Ketika  $h$  dibagi dua berulang kali, galat beda pusat menunjukkan orde mendekati dua sebelum efek pembulatan muncul.

## 5.4 Program MATLAB

```
%% Praktikum 4: beda hingga dan orde akurasi
clear; clc; format long g
f=@(x) exp(x)./sin(sqrt(x)); x0=1.2;
s=sqrt(x0);
dexact=f(x0).*(1-cot(s)./(2*s));
hs=0.2./2.^(0:7);
for k=1:numel(hs)
    h=hs(k);
    df= numdiff(f,x0,'forward',h);
    db= numdiff(f,x0,'backward',h);
    dc= numdiff(f,x0,'central',h);
    tab(k,:)=[h,df,abs(df-dexact),db,abs(db-dexact),dc,abs(dc-dexact)];
end
disp(array2table(tab,'VariableNames',{ 'h','forward','errF','backward','errB','central','errC'}));
figure; loglog(hs,tab(:,3),'o-',hs,tab(:,5),'s-',hs,tab(:,7),'^-'); grid on
xlabel('h'); ylabel('error_absolut'); legend('Forward','Backward','Central');
```



```
function d=numdiff(f,x0,method,h)
if nargin<4 || isempty(h), h=eps^(1/3)*max(1,abs(x0)); end
switch lower(method)
case 'forward', d=(f(x0+h)-f(x0))/h;
case 'backward', d=(f(x0)-f(x0-h))/h;
case 'central', d=(f(x0+h)-f(x0-h))/(2*h);
otherwise, error('Metode_tidak_dikenal. ');
end
end
```

Checklist analisis. Jangan hanya melaporkan nilai turunan. Tunjukkan hubungan  $h$ -galat dan jawab: skema mana paling akurat pada biaya evaluasi fungsi yang sebanding?

## 5.5 Latihan

- i. Turunkan sendiri formula beda pusat dari deret Taylor.
- ii. Uji  $f(x) = \sin(1/x)$  pada  $x = 0.4$  untuk beberapa  $h$ . Jelaskan mengapa fungsi yang berosilasi cepat menuntut kehati-hatian.
- iii. Estimasi orde observasi dengan  $p \approx \log(E_h/E_{h/2})/\log 2$ .
- iv. Buat fungsi untuk hampiran turunan kedua dengan beda pusat dan uji pada  $f(x) = e^x$ .
- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 6

# Praktikum 5: Ukuran Langkah, Galat Pemotongan, dan Richardson

### Capaian pembelajaran praktikum

Mahasiswa mampu menganalisis pengaruh ukuran langkah dan galat pemotongan pada turunan numerik serta mereproduksi hasil komputasi dari sumber ilmiah (indikator 6.1.7 dan 7.1.1).

### Target luaran

Eksperimen konvergensi yang menunjukkan orde metode, ekstrapolasi Richardson, dan pembahasan wilayah optimum ukuran langkah.

## 6.1 Materi singkat dan landasan teori

Misalkan aproksimasi  $D(h)$  memiliki ekspansi galat

$$D(h) = D^* + Ch^p + O(h^{p+1}), \quad (6.1)$$

dengan  $p$  orde utama. Evaluasi pada  $h$  dan  $h/2$  dapat dikombinasikan untuk mengeliminasi suku  $Ch^p$ :

$$D_R = D(h/2) + \frac{D(h/2) - D(h)}{2^p - 1}. \quad (6.2)$$

Ini adalah ide Richardson ekstrapolasi. Untuk beda pusat  $p = 2$ , penyebutnya  $2^2 - 1 = 3$ . Metode ini meningkatkan orde tanpa harus menurunkan formula baru dari awal.

Analisis konvergensi eksperimental sangat penting pada artikel komputasi. Jika solusi eksak  $D^*$  tersedia, hitung  $E_h = |D(h) - D^*|$ . Jika tidak tersedia, bandingkan perhalusan berturut-turut atau solusi referensi yang jauh lebih halus. Orde observasi dapat diperkirakan

$$p_{obs} = \frac{\log(E_h/E_{h/2})}{\log 2}. \quad (6.3)$$



Nilai  $p_{obs}$  mendekati orde teori hanya pada rentang asimtotik. Jika terlalu kasar, higher-order terms masih dominan; jika terlalu kecil, pembulatan dapat merusak pola.

## 6.2 Algoritma

- i. Pilih skema dasar dengan orde  $p$  dan urutan  $h_j = h_0/2^j$ .
- ii. Hitung  $D(h_j)$  dan  $D(h_j/2)$ .
- iii. Bentuk  $D_R$  dan galat terhadap referensi.
- iv. Plot galat terhadap  $h$  secara log-log dan hitung  $p_{obs}$ .
- v. Tandai wilayah di mana galat berhenti turun.

## 6.3 Contoh

Untuk fungsi pada Praktikum 4 dan  $x_0 = 1.2$ , beda pusat dengan  $h = 0.2$  memberi sekitar 2.87423645 dan dengan  $h = 0.1$  sekitar 2.86120081. Richardson menghasilkan sekitar 2.85685560, sangat dekat dengan nilai analitik 2.85683778.

## 6.4 Program MATLAB

```
%% Praktikum 5: Richardson dan studi step size
clear; clc; format long g
f=@(x) exp(x)./sin(sqrt(x)); x0=1.2;
s=sqrt(x0); dexact=f(x0).*(1-cot(s)./(2*s));
hs=0.4./2.^(0:8);
for k=1:numel(hs)
    h=hs(k); D1=numdiff(f,x0,'central',h); D2=numdiff(f,x0,'central',h/2);
    DR=D2+(D2-D1)/(2^2-1); % central difference berorde p=2
    T(k,:)= [h,D1,D2,DR,abs(DR-dexact)];
end
disp(array2table(T,'VariableNames',{ 'h','Dh','Dh2','Richardson','error' }));
figure; loglog(hs,T(:,5),'o-'); grid on; xlabel('h'); ylabel('error_Richardson');
```

## 6.5 Latihan

- i. Terapkan Richardson pada beda maju ( $p = 1$ ) dan bandingkan dengan beda pusat biasa.
- ii. Gunakan  $h$  dari  $10^{-1}$  sampai  $10^{-12}$ . Identifikasi titik ketika pembulatan mulai mendominasi.
- iii. Ambil satu figur/tabel turunan numerik dari artikel dan reproduksi minimal satu hasilnya. Catat semua parameter numerik yang harus diasumsikan karena tidak dilaporkan.



- iv. Diskusikan mengapa reproduksi hasil merupakan bagian dari verifikasi, bukan otomatis validasi model.
- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 7

# Praktikum 6: Integral Numerik Dasar

Capaian pembelajaran praktikum

Mahasiswa mampu menerapkan aturan Trapezoid dan Simpson serta mengevaluasi galat integrasi numerik (indikator 6.1.8).

Target luaran

Implementasi Trapezoid, Titik Tengah, dan Simpson komposit, studi jejaring perhalusan, serta perbandingan orde galat.

### 7.1 Materi singkat dan landasan teori

Integral tentu dapat dihampiri dengan mengintegrasikan interpolan polinomial. Aturan Trapezoid memakai interpolan linear pada setiap subinterval. Untuk jejaring seragam  $h = (b - a)/n$ ,

$$I_T = h \left[ \frac{1}{2} f(x_0) + \sum_{i=1}^{n-1} f(x_i) + \frac{1}{2} f(x_n) \right], \quad (7.1)$$

dengan galat global  $O(h^2)$  untuk fungsi cukup mulus.

Aturan Titik Tengah juga berorde dua:

$$I_M = h \sum_{i=1}^n f \left( a + \left( i - \frac{1}{2} \right) h \right). \quad (7.2)$$

Aturan Simpson 1/3 mengintegrasikan interpolan kuadrat dan, untuk  $n$  genap,

$$I_S = \frac{h}{3} \left[ f_0 + f_n + 4 \sum_{i \text{ ganjil}} f_i + 2 \sum_{i \text{ genap}, i \neq 0, n} f_i \right], \quad (7.3)$$

dengan galat global  $O(h^4)$ .





## 7.2 Algoritma

- i. Pilih  $n$ , hitung  $h = (b - a)/n$  dan titik grid.
- ii. Evaluasi  $f$  secara vektor bila memungkinkan.
- iii. Terapkan bobot sesuai metode.
- iv. Ulangi untuk  $n, 2n, 4n, \dots$  dan amati penurunan galat.
- v. Pastikan  $n$  genap untuk Simpson 1/3 komposit.

## 7.3 Contoh

Untuk  $\int_0^\pi \sin x \, dx = 2$ , gunakan  $n = 4, 8, 16, \dots$ . Trapezoid dan Titik Tengah menunjukkan galat berorde dua, sedangkan Simpson turun jauh lebih cepat karena berorde empat pada fungsi mulus.

## 7.4 Program MATLAB

```
%% Praktikum 6: Trapezoid, Midpoint, Simpson
clear; clc; format long g
f=@(x) sin(x); a=0; b=pi; Iexact=2;
ns=[4 8 16 32 64];
for k=1:numel(ns)
    n=ns(k);
    It=trap_comp(f,a,b,n); Im=midpoint_comp(f,a,b,n); Is=simpson_comp(f,a,b,n);
    T(k,:)=[n, It, abs(It-Iexact), Im, abs(Im-Iexact), Is, abs(Is-Iexact)];
end
disp(array2table(T, 'VariableNames', {'n', 'Trap', 'errT', 'Mid', 'errM', 'Simp', 'errS'}));

function I=trap_comp(f,a,b,n)
h=(b-a)/n; x=linspace(a,b,n+1); y=f(x);
I=h*(0.5*y(1)+sum(y(2:end-1))+0.5*y(end));
end

function I=midpoint_comp(f,a,b,n)
h=(b-a)/n; xm=a+h*((1:n)-0.5); I=h*sum(f(xm));
end

function I=simpson_comp(f,a,b,n)
if mod(n,2)~=0, error('Simpson_1/3_komposit_memerlukan_n_genap. '); end
h=(b-a)/n; x=linspace(a,b,n+1); y=f(x);
I=h/3*(y(1)+y(end)+4*sum(y(2:2:end-1))+2*sum(y(3:2:end-2)));
end
```

Catatan numerik. Aturan dengan orde tinggi tidak selalu unggul pada data tidak mulus atau noisy. Asumsi regularitas fungsi harus dipertimbangkan sebelum menyimpulkan metode terbaik.



## 7.5 Latihan

- i. Evaluasi  $\int_0^{\pi/2} (6 + 3 \cos x) dx$  dengan beberapa  $n$  dan hitung galat relatif.
- ii. Terapkan Trapezoid pada data tabular  $x = [0, 0.1, \dots, 0.5]$  dengan  $f = [1, 8, 4, 3.5, 5, 1]$ .
- iii. Buat grafik galat terhadap  $n$  dan estimasikan orde observasi.
- iv. Jelaskan kondisi ketika data tabular membuat Simpson tidak dapat langsung diterapkan.
- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 8

# Praktikum 7: Titik Tengah dan Romberg

Capaian pembelajaran praktikum

Mahasiswa mampu menerapkan Titik Tengah dan Romberg serta membandingkan akurasi dengan metode integrasi lain; mampu mereproduksi hasil integrasi numerik dari artikel (indikator 6.1.9 dan 7.1.1).

Target luaran

Tabel Romberg, perbandingan Trapezoid/Titik Tengah/Simpson/Romberg, dan analisis biaya terhadap akurasi.

### 8.1 Materi singkat dan landasan teori

Romberg menggabungkan perhalusan berturut-turut Trapezoid dengan Richardson ekstrapolasi. Kolom pertama adalah

$$R_{j,1} = T(h_j), \quad h_j = \frac{b-a}{2^{j-1}}. \quad (8.1)$$

Kolom selanjutnya dibentuk rekursif

$$R_{j,k} = R_{j,k-1} + \frac{R_{j,k-1} - R_{j-1,k-1}}{4^{k-1} - 1}, \quad k \leq j. \quad (8.2)$$

Karena galat utama Trapezoid mengandung pangkat genap  $h^2, h^4, \dots$ , tiap ekstrapolasi menghilangkan satu suku galat utama. Tabel Romberg juga memberikan cara alami memantau konvergensi tanpa solusi analitik: bandingkan elemen diagonal successive.

Titik Tengah berguna sebagai pembanding karena memakai titik interior dan memiliki galat  $O(h^2)$  dengan konstanta berbeda dari Trapezoid. Dalam praktik, metode terbaik dipilih berdasarkan kemulusan, biaya evaluasi fungsi, dan target akurasi, bukan orde saja.

### 8.2 Algoritma Romberg




---

### Algorithm 5 Integrasi Romberg

---

- 1: for  $j = 1, \dots, m$  do
  - 2:     Hitung  $R_{j,1} = T$  dengan  $n = 2^{j-1}$  subinterval.
  - 3:     for  $k = 2, \dots, j$  do
  - 4:          $R_{j,k} \leftarrow R_{j,k-1} + [R_{j,k-1} - R_{j-1,k-1}] / (4^{k-1} - 1)$ .
  - 5:     end for
  - 6: end for
  - 7: Ambil  $R_{m,m}$  atau hentikan ketika diagonal sudah memenuhi toleransi.
- 

## 8.3 Contoh

Hampiri

$$I = \int_0^2 \frac{e^x \sin x}{1+x^2} dx. \quad (8.3)$$

Nilai referensi numerik presisi tinggi sekitar 1.94013002203. Tabel Romberg memperlihatkan bagaimana perhalusan berturut-turut dan ekstrapolasi menghasilkan konvergensi cepat tanpa langsung memanggil penyelesaian numerik bawaan sebagai solusi utama.

## 8.4 Program MATLAB

```
%% Praktikum 7: Romberg
clear; clc; format long g
f=@(x) exp(x).*sin(x)./(1+x.^2); a=0; b=2;
[R,I]=romberg_fun(f,a,b,6);
Iref=integral(f,a,b,'AbsTol',1e-13,'RelTol',1e-13);
disp(R); fprintf('Romberg=%.12f, referensi=%.12f, error=%.3e\n',I,Iref,abs(I-Iref));

function [R,I]=romberg_fun(f,a,b,m)
R=nan(m,m);
for j=1:m
    n=2^(j-1); R(j,1)=trap_comp(f,a,b,n);
    for k=2:j
        R(j,k)=R(j,k-1)+(R(j,k-1)-R(j-1,k-1))/(4^(k-1)-1);
    end
end
I=R(m,m);
end
```

Checklist analisis. Bandingkan bukan hanya galat akhir, tetapi juga jumlah evaluasi  $f$ . Penyelesai numerik dengan galat kecil tetapi evaluasi sangat banyak belum tentu efisien.



## 8.5 Latihan

- i. Gunakan Romberg untuk  $\int_0^3 xe^{2x} dx$  dan bandingkan dengan solusi analitik.
- ii. Buat kriteria berhenti berdasarkan  $|R_{j,j} - R_{j-1,j-1}|$ .
- iii. Bandingkan Trapezoid, Titik Tengah, Simpson, dan Romberg pada fungsi dengan perubahan tajam.
- iv. Reproduksi satu hasil integral numerik dari artikel. Laporkan grid, toleransi, dan penyelesai numerik pembandingan.
- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 9

# Praktikum 8: Masalah Nilai Awal - Euler dan Runge-Kutta

### Capaian pembelajaran praktikum

Mahasiswa mampu menerapkan metode Euler untuk masalah nilai awal dan mengevaluasi galat lokal/global (indikator 6.1.10), serta memperoleh pengantar implementasi Runge-Kutta untuk perbandingan.

### Target luaran

Penyelesai numerik Euler dan RK4 modular, kurva solusi, galat terhadap solusi referensi, dan studi ukuran langkah.

## 9.1 Materi singkat dan landasan teori

Masalah nilai awal berbentuk

$$y'(t) = f(t, y), \quad y(t_0) = y_0. \quad (9.1)$$

Ekspansi Taylor satu langkah memberi

$$y(t+h) = y(t) + hy'(t) + O(h^2), \quad (9.2)$$

sehingga Euler eksplisit

$$y_{n+1} = y_n + hf(t_n, y_n). \quad (9.3)$$

Galat pemotongan lokal per langkah adalah  $O(h^2)$ , sedangkan galat global setelah banyak langkah adalah  $O(h)$ . Ini menjelaskan mengapa pembagian ukuran langkah menjadi dua idealnya mengurangi galat global kira-kira faktor dua pada rentang asimtotik.

Runge-Kutta orde empat (RK4) mencapai akurasi global  $O(h^4)$  tanpa membutuhkan turunan eksplisit  $f$ . Ia menggabungkan empat estimasi kemiringan:

$$k_1 = f(t_n, y_n), \quad (9.4)$$



$$k_2 = f(t_n + h/2, y_n + hk_1/2), \quad (9.5)$$

$$k_3 = f(t_n + h/2, y_n + hk_2/2), \quad (9.6)$$

$$k_4 = f(t_n + h, y_n + hk_3), \quad (9.7)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (9.8)$$

## 9.2 Contoh model

Model logistik

$$y' = ry \left(1 - \frac{y}{K}\right), \quad y(0) = y_0 \quad (9.9)$$

memiliki solusi analitik

$$y(t) = \frac{K}{1 + (K/y_0 - 1)e^{-rt}}. \quad (9.10)$$

Dengan  $K = 100$ ,  $r = 0.4$ , dan  $y_0 = 10$ , nilai pada  $t = 10$  sekitar 85.8486. Solusi analitik menjadi verifikasi case untuk menguji implementasi Euler/RK4.

## 9.3 Algoritma

- i. Bentuk grid  $t_n = t_0 + nh$ .
- ii. Tetapkan  $y_0$ .
- iii. Untuk tiap langkah, evaluasi formula Euler atau RK4.
- iv. Simpan seluruh pasangan  $(t_n, y_n)$ .
- v. Jika referensi tersedia, hitung galat maksimum dan ulangi untuk beberapa  $h$ .

## 9.4 Program MATLAB

```
%% Praktikum 8: Euler dan RK4 pada model logistik
clear; clc; format long g
K=100; r=0.4; y0=10; f=@(t,y) r*y*(1-y/K);
tspan=[0 10]; h=0.5;
[tE,yE]=euler_ivp(f,tspan,y0,h);
[tR,yR]=rk4_ivp(f,tspan,y0,h);
yexact=@(t) K./(1+(K/y0-1).*exp(-r*t));
errE=max(abs(yE-yexact(tE))); errR=max(abs(yR-yexact(tR)));
fprintf('Max_error_Euler=%.3e, RK4=%.3e\n',errE,errR);
figure; plot(tE,yexact(tE),'k-',tE,yE,'o-',tR,yR,'s-'); grid on
xlabel('t'); ylabel('y'); legend('Eksak','Euler','RK4','Location','best');
```



```
function [t,y]=euler_ivp(f,tspan,y0,h)
t=(tspan(1):h:tspan(2))'; if t(end)<tspan(2), t=[t;tspan(2)]; end
y=zeros(numel(t),numel(y0)); y(1,:)=y0(:)';
for n=1:numel(t)-1
    hn=t(n+1)-t(n); y(n+1,:)=y(n,:)+hn*f(t(n),y(n,:))';
end
end

function [t,y]=rk4_ivp(f,tspan,y0,h)
t=(tspan(1):h:tspan(2))'; if t(end)<tspan(2), t=[t;tspan(2)]; end
y=zeros(numel(t),numel(y0)); y(1,:)=y0(:)';
for n=1:numel(t)-1
    hn=t(n+1)-t(n); yn=y(n,:); tn=t(n);
    k1=f(tn,yn); k2=f(tn+hn/2,yn+hn*k1/2);
    k3=f(tn+hn/2,yn+hn*k2/2); k4=f(tn+hn,yn+hn*k3);
    y(n+1,:)=(yn+hn*(k1+2*k2+2*k3+k4)/6)';
end
end
```

## 9.5 Latihan

- i. Uji  $h = 1, 0.5, 0.25, 0.125$  pada model logistik dan estimasikan orde global Euler serta RK4.
- ii. Terapkan Euler pada  $y' = -2y + t$ ,  $y(0) = 1$ . Bandingkan dengan solusi analitik.
- iii. Modifikasi penyelesaian numerik agar dapat menangani sistem ODE dua variabel.
- iv. Jelaskan perbedaan galat pemotongan lokal dan galat global dengan kalimat Anda sendiri.
- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.





## Bab 10

# Praktikum 9: Stabilitas, Ukuran Langkah, dan Reproduksi Model PDB

### Capaian pembelajaran praktikum

Mahasiswa mampu menerapkan Runge-Kutta dan menganalisis akurasi, stabilitas, serta pengaruh ukuran langkah; mampu mereproduksi kurva/model sederhana dari artikel (indikator 6.1.11 dan 7.1.1).

### Target luaran

Studi stabilitas ukuran langkah pada persamaan uji, perbandingan Euler-RK4, serta satu reproduksi kurva model dari literatur.

## 10.1 Materi singkat dan landasan teori

Akurasi tidak cukup: metode ODE juga harus stabil. Persamaan uji

$$y' = \lambda y, \quad \Re(\lambda) < 0 \quad (10.1)$$

memiliki solusi yang meluruh. Euler memberikan

$$y_{n+1} = (1 + h\lambda)y_n. \quad (10.2)$$

Agar solusi numerik juga meluruh, diperlukan

$$|1 + h\lambda| < 1. \quad (10.3)$$

Untuk  $\lambda$  real negatif, syarat ini menjadi  $0 < h < -2/\lambda$ . Jadi ukuran langkah yang terlalu besar dapat menghasilkan osilasi/growth numerik walaupun solusi kontinu stabil.

RK4 memiliki polinom stabilitas

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \quad z = h\lambda. \quad (10.4)$$



Region  $|R(z)| < 1$  lebih luas daripada Euler, tetapi tetap terbatas. Pada masalah kaku, metode eksplisit dapat memerlukan step sangat kecil demi stabilitas, bukan demi akurasi.

Reproduksi artikel harus membedakan data/model parameter dari parameter numerik. Dua simulasi dengan model dan parameter fisik sama dapat berbeda jika penyelesai numerik, toleransi, atau ukuran langkah tidak sama.

## 10.2 Contoh

Ambil  $y' = -15y$ ,  $y(0) = 1$ . Euler dengan  $h = 0.2$  memiliki faktor amplifikasi  $1 - 3 = -2$ , sehingga tidak stabil. Dengan  $h = 0.1$ , faktor  $-0.5$  dan iterasi stabil tetapi berosilasi tanda. Ini merupakan contoh sederhana bahwa ukuran langkah adalah bagian dari kualitas solusi.

## 10.3 Program MATLAB

```
%% Praktikum 9: stabilitas dan pengaruh step size
clear; clc; format long g
lambda=-15; f=@(t,y) lambda*y; y0=1; tspan=[0 1];
hs=[0.2 0.1 0.05 0.025];
for k=1:numel(hs)
    h=hs(k); [tE,yE]=euler_ivp(f,tspan,y0,h); [tR,yR]=rk4_ivp(f,tspan,y0,h);
    exact=exp(lambda*tE);
    T(k,:)=[h,max(abs(yE-exact)),max(abs(yR-exact))];
end
disp(array2table(T,'VariableNames',{ 'h', 'errEuler', 'errRK4' }));
% Euler stabil untuk masalah uji y'=lambda*y jika |1+h*lambda|<1.
```

Checklist analisis. Pada reproduksi artikel, catat: model, parameter, kondisi awal, interval waktu, penyelesai numerik, ukuran langkah/tolerance, dan pengolahan akhir. Jika salah satu tidak tersedia, nyatakan asumsi secara eksplisit.

## 10.4 Latihan

- Untuk  $\lambda = -5, -15, -50$ , tentukan batas stabilitas Euler secara analitik lalu uji dengan kode.
- Buat grafik fungsi stabilitas  $|1 + z|$  sepanjang sumbu real negatif dan tandai interval stabil Euler.
- Reproduksi satu kurva ODE dari artikel/model yang Anda pilih. Lakukan jejaring perhalusan minimal dua kali.
- Bandingkan hasil RK4 manual dengan ode45 sebagai penyelesai numerik pembanding, bukan sebagai pengganti analisis.



- v. Ulangi percobaan dengan toleransi atau ukuran langkah yang berbeda, lalu jelaskan apakah kesimpulan berubah.
- vi. Tambahkan satu grafik pendukung dan beri judul, label sumbu, legenda, serta penjelasan singkat.
- vii. Susun satu paragraf interpretasi hasil: apa yang berhasil, apa keterbatasannya, dan parameter numerik mana yang paling berpengaruh.



## Bab 11

# Praktikum 10: Praktikum Terpadu, Verifikasi, dan Validasi

### Capaian pembelajaran praktikum

Mahasiswa mampu memilih metode numerik yang sesuai, mengintegrasikan beberapa metode dalam satu alur kerja, memeriksa keakuratan dan keterbatasan hasil, serta mendokumentasikan proses komputasi secara ilmiah (penguatan CPMK 6.1 dan 7.1).

### Target luaran

Satu mini-pipeline komputasi yang menghubungkan formulasi masalah, penyelesai numerik, verifikasi, analisis sensitivitas numerik, dan interpretasi luaran sebagai persiapan UAS dan proyek.

### 11.1 Materi singkat: dari metode ke alur kerja

Pada masalah nyata, metode numerik jarang berdiri sendiri. Sebuah model dapat memerlukan pencarian akar untuk event time, penyelesaian SPL di dalam iterasi nonlinear, turunan/integral untuk kuantitas turunan, dan penyelesai numerik ODE untuk dinamika. Karena itu mahasiswa perlu beralih dari pertanyaan “rumus apa?” menjadi “alur kerja apa yang dapat dipertanggungjawabkan?”.

Alur kerja yang disarankan:

- Formulasi: definisikan variabel, parameter, persamaan, domain, kondisi awal/batas, dan kuantitas yang ingin dihitung.
- Pemilihan metode: cocokkan struktur masalah dengan penyelesai numerik dan pahami asumsi metode.
- Implementasi: gunakan fungsi modular, parameter numerik eksplisit, dan penyimpanan history.



- iv. Verifikasi: gunakan solusi analitik, masalah uji terancang, residual, pemeriksaan konservasi, atau jejaring perhalusan.
- v. Validasi: bila data tersedia, bandingkan model dengan data/fenomena menggunakan metrik yang relevan.
- vi. Sensitivitas numerik: ubah  $h$ , toleransi, tebakan awal, dan penyelesai numerik untuk memastikan kesimpulan tidak merupakan artefak numerik.
- vii. Pelaporan: simpan tabel, grafik, kode, konfigurasi, dan interpretasi.

## 11.2 Contoh terpadu: model logistik

Gunakan model logistik pada Praktikum 8. Pertanyaan komputasi dapat dirangkai sebagai berikut.

- i. Kapan  $y(t)$  pertama kali mencapai 80? Ini menjadi masalah akar  $q(t) = y(t) - 80 = 0$  dan dapat diselesaikan dengan Biseksi. Nilai referensinya sekitar  $t = 8.9588$ .
- ii. Bagaimana kurva dinamikanya? Gunakan RK4 dan bandingkan dengan solusi analitik.
- iii. Berapa laju perubahan di  $t = 5$ ? Gunakan turunan numerik sebagai pemeriksaan silang terhadap ruas kanan ODE.
- iv. Berapa  $\int_0^{10} y(t) dt$ ? Gunakan Simpson sebagai ukuran akumulasi; nilai referensi sekitar 462.50.
- v. Apakah hasil stabil jika  $h$  RK4 dibagi dua dan toleransi akar diperketat?

## 11.3 Algoritma alur kerja

---

### Algorithm 6 Mini-pipeline numerik

---

- 1: Definisikan model dan parameter.
  - 2: Tentukan masalah uji/solusi acuan untuk verifikasi bila tersedia.
  - 3: Selesaikan event/root quantity.
  - 4: Simulasikan dinamika dengan minimal satu penyelesai numerik utama.
  - 5: Hitung derived quantities (turunan/integral) dengan metode yang sesuai.
  - 6: Lakukan perhalusan/sensitivity numerik.
  - 7: Kumpulkan residual, galat, grafik, dan tabel.
  - 8: Interpretasikan hasil dan nyatakan keterbatasan.
-



## 11.4 Program MATLAB

```
%% Praktikum 10: pipeline terintegrasi dan verification & validation
clear; clc; format long g
K=100; r=0.4; y0=10;
yexact=@(t) K./(1+(K/y0-1).*exp(-r*t));
% 1) Pencarian akar: kapan populasi mencapai 80?
q=@(t) yexact(t)-80;
[t80,hroot]=bisection_fun(q,0,20,1e-10,100);
% 2) Simulasi model dengan RK4
f=@(t,y) r*y*(1-y/K); [t,y]=rk4_ivp(f,[0 10],y0,0.25);
% 3) Turunan numerik pada t=5 dari kurva analitik sebagai verification case
dy5=numdiff(yexact,5,'central',1e-3);
% 4) Integral numerik: area di bawah kurva 0<=t<=10
I=simpson_comp(yexact,0,10,100);
% 5) Residual verifikasi solver terhadap solusi analitik
err=max(abs(y-yexact(t)));
fprintf('t_saat_y=80: %.8f\n',t80);
fprintf('dy/dt_pada_t=5: %.8f\n',dy5);
fprintf('Integral_y_dt: %.8f\n',I);
fprintf('Max_error_RK4: %.3e\n',err);
```

## 11.5 Latihan terpadu

- Buat tabel yang memetakan tipe masalah ke metode yang tersedia pada Praktikum 1–9. Tambahkan kelebihan, keterbatasan, dan parameter numerik utama.
- Pilih satu model sederhana (peluruhan, logistik, pendinginan Newton, RC circuit, atau model lain). Bangun alur kerja yang memuat minimal tiga metode numerik berbeda.
- Lakukan verifikasi dengan solusi analitik atau manufactured solution. Jika tidak ada, gunakan jejaring/tolerance perhalusan dan jelaskan keterbatasannya.
- Buat satu halaman catatan keterulangan: versi MATLAB, nama file, parameter, seed (jika ada bilangan acak), dan langkah menjalankan program.



## Rubrik Analisis Umum

| Aspek        | Pertanyaan yang harus dapat dijawab                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Formulasi    | Apa persamaan, variabel, domain, asumsi, dan kuantitas yang dicari?                                                                 |
| Metode       | Mengapa metode ini sesuai? Apa syarat/risiko kegagalannya?                                                                          |
| Implementasi | Apakah program modular, dapat dijalankan ulang, dan tidak memakai operasi numerik yang buruk seperti invers eksplisit tanpa alasan? |
| Konvergensi  | Apa kriteria berhenti? Bagaimana residual/galat berubah?                                                                            |
| Akurasi      | Apa solusi pembandingan? Bagaimana orde/ukuran langkah mempengaruhi galat?                                                          |
| Ketahanan    | Apa yang terjadi jika tebakan awal, toleransi, atau ukuran langkah diubah?                                                          |
| Interpretasi | Apa arti angka/kurva dalam konteks model?                                                                                           |
| Keterulangan | Dapatkah orang lain menjalankan kode dan memperoleh hasil yang sama?                                                                |



## Daftar File MATLAB

Paket kode menyertakan skrip utama dan fungsi berikut.

- root\_compare.m
- bisection\_fun.m
- regula\_falsi\_fun.m
- newton\_fun.m
- secant\_fun.m
- fixedpoint\_fun.m
- spl\_iterative.m
- jacobi\_iter.m
- gauss\_seidel\_iter.m
- spnl\_curvefit.m
- newton\_system.m
- gauss\_newton\_damped.m
- finite\_difference.m
- numdiff.m
- richardson\_diff.m
- integration\_basic.m
- trap\_comp.m
- midpoint\_comp.m
- simpson\_comp.m
- romberg\_demo.m
- romberg\_fun.m
- ode\_euler\_rk4.m
- euler\_ivp.m
- rk4\_ivp.m
- ode\_stability.m
- integrated\_review.m





## Referensi Utama

- i. Chapra, S. C. dan Canale, R. P. (2011). Numerical Methods for Engineers. McGraw-Hill.
- ii. Press, W. H., Vetterling, W. T., Teukolsky, S. A., dan Flannery, B. P. (1986). Numerical Recipes. Cambridge University Press.
- iii. Moore, H. dan Sanadhya, S. (2007). MATLAB for Engineers. Prentice Hall.
- iv. Anton, H. dan Rorres, C. (2013). Elementary Linear Algebra: Applications Version. John Wiley & Sons.
- v. Wiryanto, L. H. (2011). Matematika Numerik.